



PYTHON CON DJANGO

Estudio

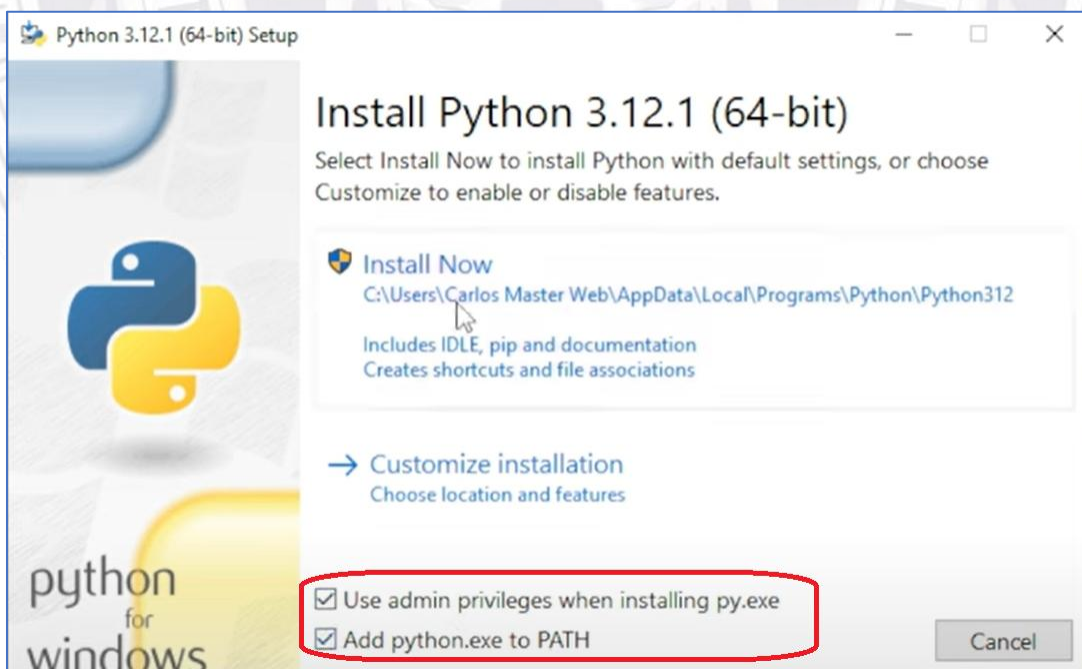
Requisitos

Instalación Python:

Se descarga e instala directamente desde la página oficial <https://www.python.org>.

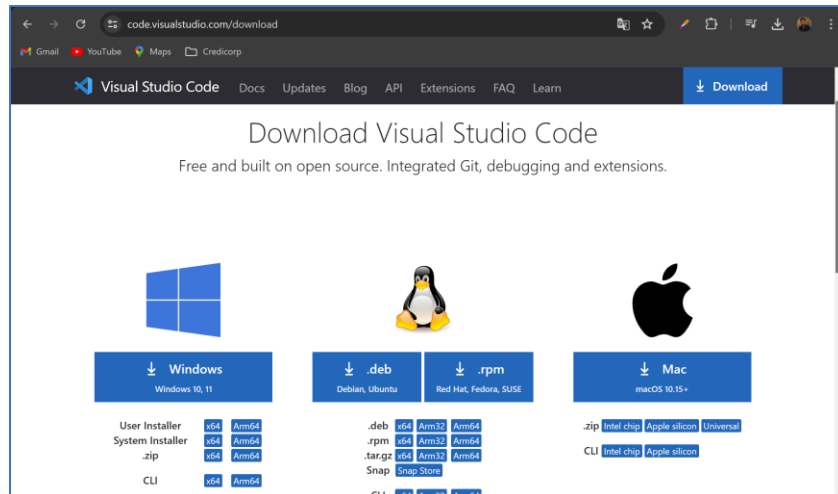


No olvide seleccionar las dos opciones de la parte inferior en el proceso de instalación.



Instalación Visual Studio Code

Se descarga e instala directamente desde la página oficial <https://code.visualstudio.com/>.



Instalación Pip

Pip es el gestor de paquetes de Python, y su instalación se realiza normalmente con el mismo Python, para asegurarnos de que está instalado y de su versión ejecute:

```
pip --version
```

En caso de no estarlo, ejecute:

```
python -m ensurepip --default-pip
```

donde:

python determina la ejecución del interprete propio del lenguaje.

-m indica que debe ejecutar un módulo como script y no un archivo.

ensurepip se asegura que pip no este instalado, y de ser así lo instala.

--default instala la versión por defecto que se adapte mejor a la versión Python.

-pip es el módulo a instalar.

Instalación Virtualenv

Lo siguiente seria la instalación de virtualenv, el cual nos ayuda a generar entornos aislados para nuestros proyectos.

```
pip install virtualenv
```

una vez creado el proyecto e instalado virtualenv, se le genera un entorno virtual y se activa (por lo general el nombre del entorno es env).

```
python -m venv [nombre entorno]  
[nombre entorno]\Scripts\activate
```

Una vez activo el entorno virtual, cualquier comando que se ejecute (como pip install) afectara solamente a el entorno del proyecto, y no al global. Para desactivarlo se debe ejecutar:

```
deactivate
```

Instalación Django

Una vez activo nuestro entorno virtual podemos ejecutar el siguiente comando para la instalación de Django:

```
pip install django
```

y para validar su instalación:

```
django-admin --version
```

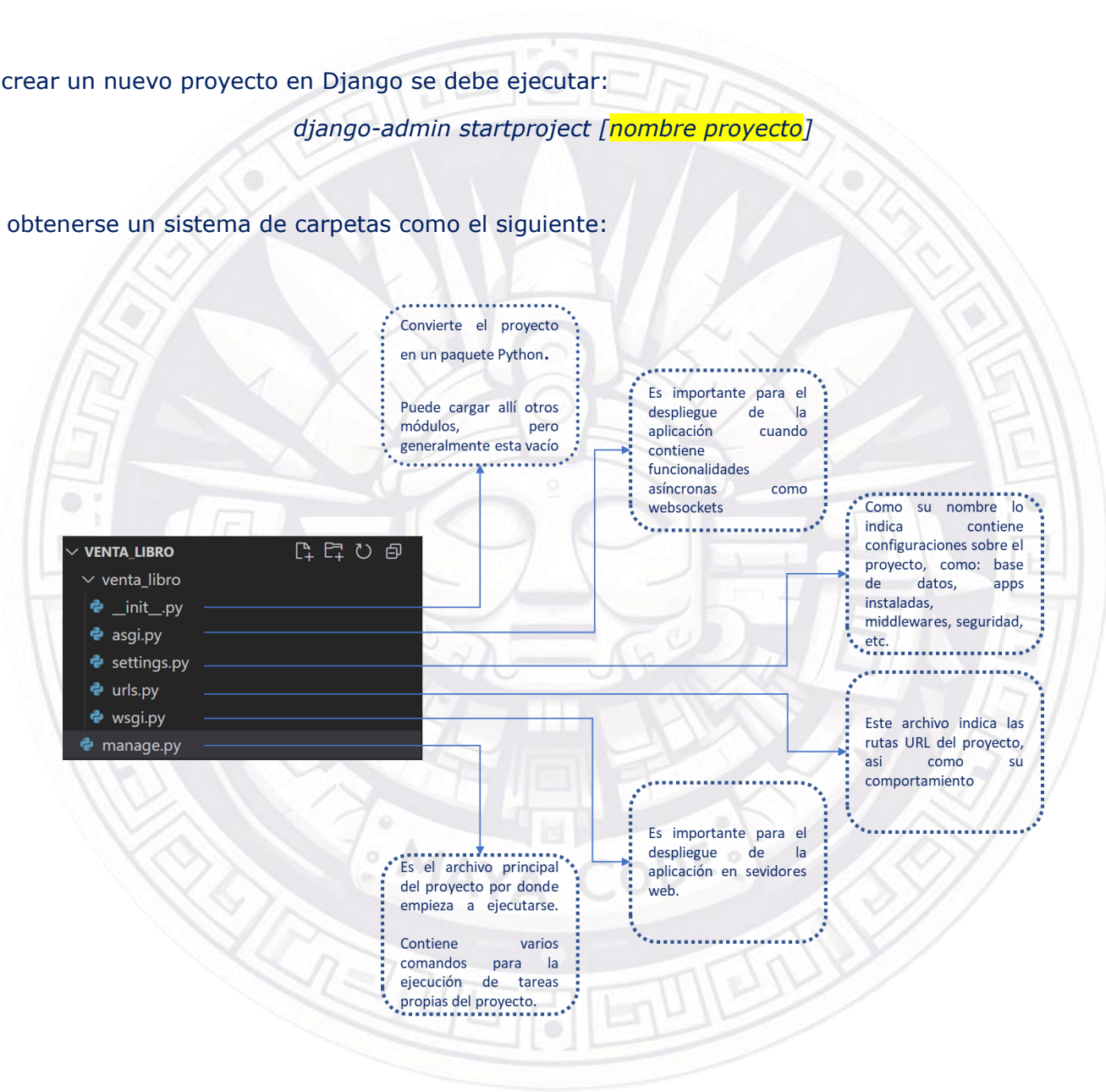
Instalarlo también en el entorno virtual del proyecto

Creación Proyecto Django

Para crear un nuevo proyecto en Django se debe ejecutar:

```
django-admin startproject [nombre proyecto]
```

debe obtenerse un sistema de carpetas como el siguiente:



Archivo requirements.txt

El archivo **requirements.txt** es un archivo de texto que contiene una lista de todas las dependencias (paquetes o bibliotecas) de un proyecto en Python, junto con sus versiones especificadas, si es necesario. Este archivo permite a los desarrolladores (incluyendo al creador) instalar las mismas versiones de las bibliotecas que está usando el proyecto, garantizando que todos trabajen con las mismas dependencias.

Para crearlo debemos ejecutar:

```
pip freeze > requirements.txt
```

y para instalar las dependencias allí inscritas:

```
pip install -r requirements.txt
```


Creación Aplicación Django

Las aplicaciones en Django, son componentes modulares que contienen funciones relacionadas.

Para crearla debemos ejecutar:

```
python manage.py startapp [Nombre aplicación]
```

debemos obtener un sistema de carpetas como:

> migrations	→ Lleva un registro de las migraciones a db
🔗 __init__.py	→ Marca la carpeta como una aplicación python
🔗 admin.py	→ Configura el panel de administración automático de django
🔗 apps.py	→ Configura la aplicación
🔗 models.py	→ Define los modelos de la aplicación
🔗 tests.py	→ Nos ayuda a implementar las pruebas unitarias
🔗 views.py	→ Implementa las lógicas de las vistas

Además, debemos crear un archivo `urls.py`, el cual nos ayudara con la definición de rutas url para la aplicación.

Luego de creada la aplicación, debe registrarse dentro del proyecto, para que este la detecte.

Ingresa al archivo `settings.py`, busque la lista `INSTALLED_APPS` y agregue el nombre de la aplicación, ejemplo:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'productos'  
]
```

Ejecución Aplicación

Para ejecutar la aplicación se debe ejecutar:

```
python manage.py runserver
```

El puerto por defecto será el 8000, si se desea un puerto específico:

```
python manage.py runserver 8080
```

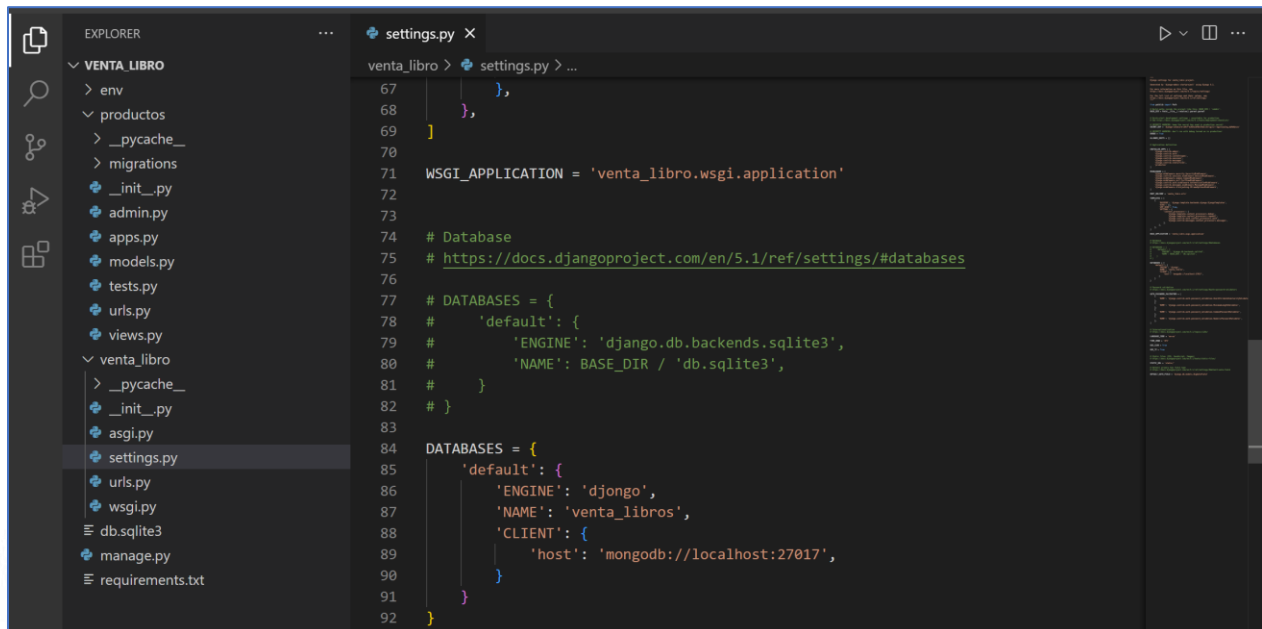
Conectar con Db Mongo

Para aplicar una conexión a base de datos mongo, se debe instalar primero los paquetes por consola, con los siguientes comandos:

```
pip install djongo
```

```
pip install pymongo
```

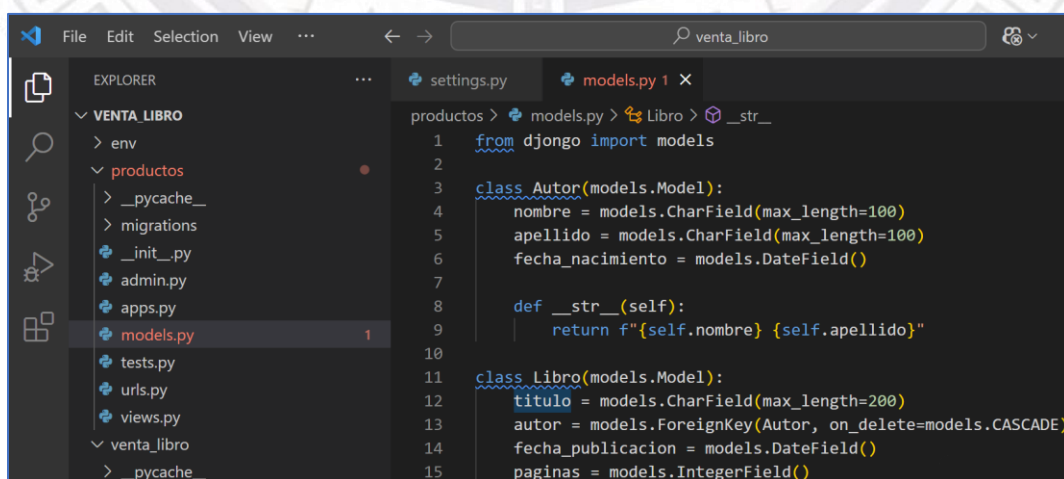
Luego, se tiene que hacer la configuración en el archivo settings.py del proyecto:



```
67     },
68 },
69 ],
70
71 WSGI_APPLICATION = 'venta_libro.wsgi.application'
72
73 # Database
74 # https://docs.djangoproject.com/en/5.1/ref/settings/#databases
75
76 # DATABASES = {
77 #     'default': {
78 #         'ENGINE': 'django.db.backends.sqlite3',
79 #         'NAME': BASE_DIR / 'db.sqlite3',
80 #     }
81 # }
82
83 DATABASES = {
84     'default': {
85         'ENGINE': 'djongo',
86         'NAME': 'venta_libros',
87         'CLIENT': {
88             'host': 'mongodb://localhost:27017',
89         }
90     }
91 }
92
```

Generacion de modelos y exportación a db

En nuestro archivo model, vamos a realizar la generación de modelos que luego serán exportados a la db que se encuentre asociada.



```
1 from django import models
2
3 class Autor(models.Model):
4     nombre = models.CharField(max_length=100)
5     apellido = models.CharField(max_length=100)
6     fecha_nacimiento = models.DateField()
7
8     def __str__(self):
9         return f"{self.nombre} {self.apellido}"
10
11 class Libro(models.Model):
12     titulo = models.CharField(max_length=200)
13     autor = models.ForeignKey(Autor, on_delete=models.CASCADE)
14     fecha_publicacion = models.DateField()
15     paginas = models.IntegerField()
```

Luego haremos la migración a base de datos:

Con el siguiente comando, preparamos la migración:

```
python manage.py makemigrations
```

y luego para ejecutar la migración escribimos en consola:

```
python manage.py migrate
```

Creación de super usuario y administración del sitio

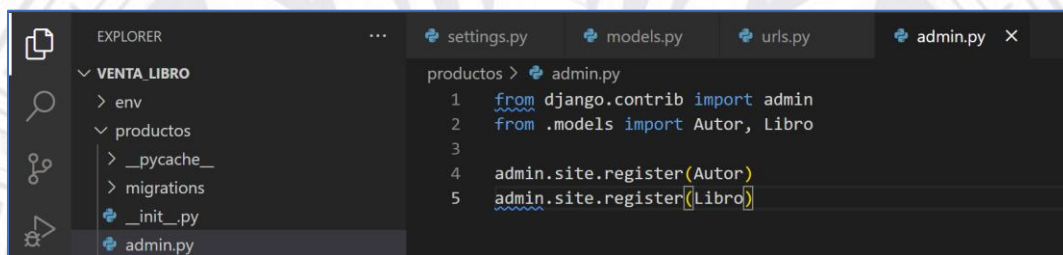
El super usuario nos ayudara a trabajar con la interfaz grafica de Django.

Para crearlo se debe ejecutar el siguiente comando:

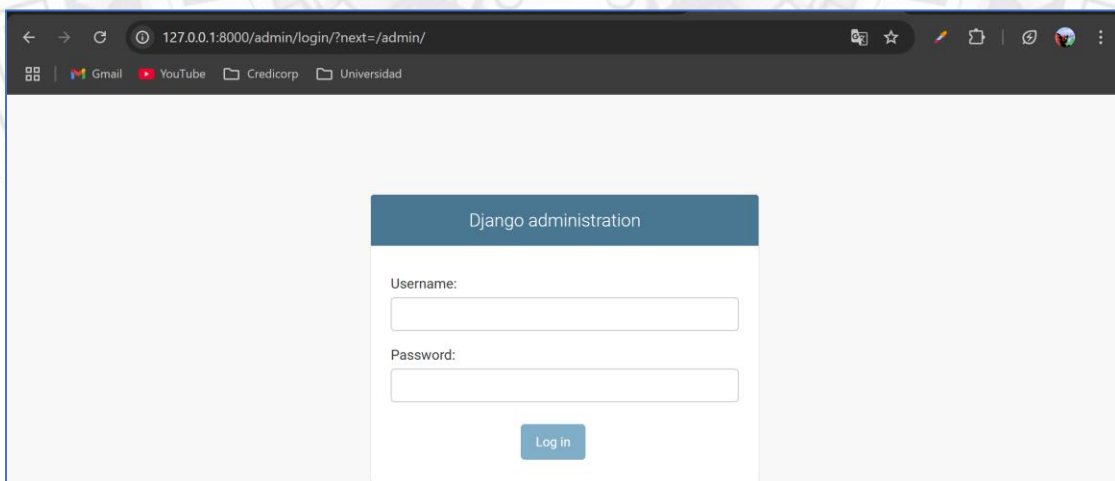
```
python manage.py createsuperuser
```

Y posterior ingresar las credenciales solicitadas.

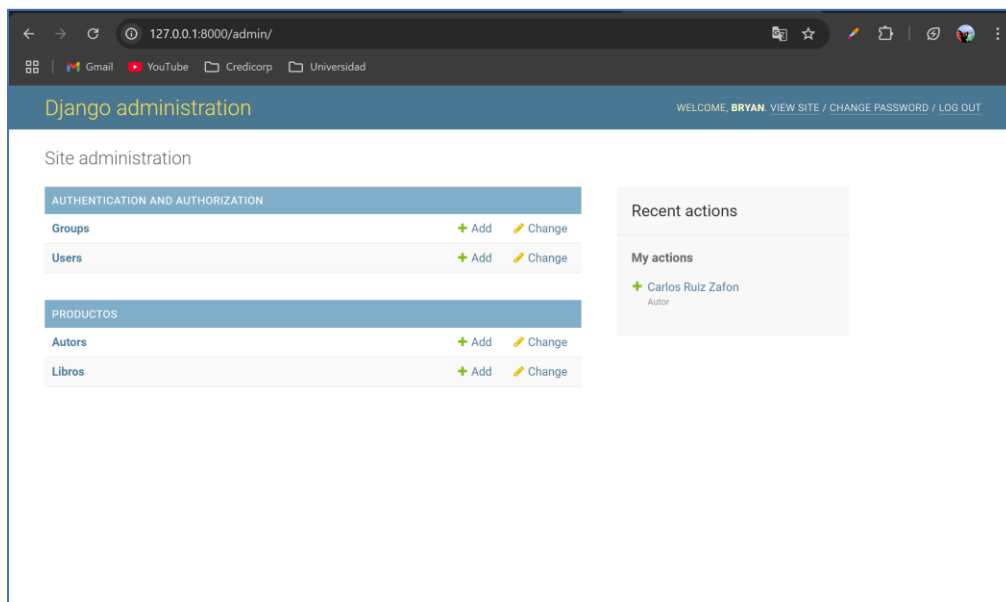
Hecho esto, debemos registrar en el archivo admin.py los modelos generados:



Para validar la correcta ejecución, corremos el proyecto e ingresamos a <http://127.0.0.1:8000/admin/> con las credenciales definidas en el proceso anterior.



Y desde allí por medio de la interfaz gráfica podremos manipular la información de las tablas.



Creación de vistas basadas en funciones (fbv) y enrutamiento

Cuando ingresamos al localhost estaríamos ejecutando directamente el proyecto, sin embargo, según nuestra estructura deseamos ejecutar las funcionalidades que están en nuestras aplicaciones, entonces, en **urls.py** del proyecto vamos a hacer el enlace a la url de la aplicación, para esto debemos importar la funcionalidad **include** y asignar un nuevo **path**.

```
venta_libro > 📄 urls.py > ...
1  from django.contrib import admin
2  from django.urls import path, include
3
4
5  urlpatterns = [
6      path('admin/', admin.site.urls),
7      path('productos/', include('productos.urls'))
8  ]
9
```

Ahora, debemos construir nuestras funciones en la aplicación. De acuerdo al esquema, estas funciones deberán estar en el archivo **views.py**.

Estas vistas recibirán una request http, y devolverán una response http.

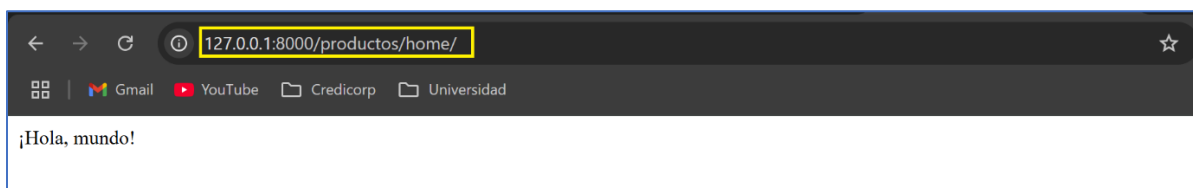
por el momento, que aun no hemos creado templates, vamos a devolver un texto puntual.

```
productos > 📄 views.py > ...
1  from django.http import HttpResponse
2
3  def inicio(request):
4      return HttpResponse("¡Hola, mundo!")
5
```

Y, por último, deberemos apuntar una url hacia la función nueva, desde **urls.py** de la aplicación.


```
productos > urls.py > ...
1  from django.urls import path
2  from .views import inicio
3
4  urlpatterns = [
5      path('home/', inicio, name='inicio')
6  ]
```

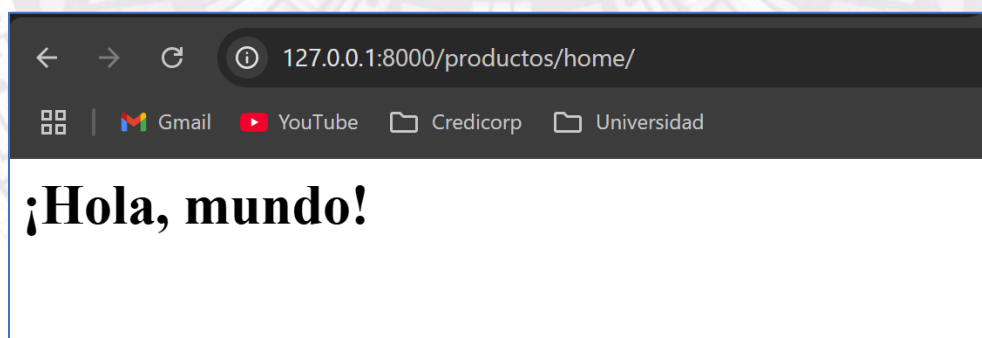
El resultado debería ser:



El sistema renderiza el string que estamos devolviendo, eso quiere decir que podemos renderizar html, por ejemplo:

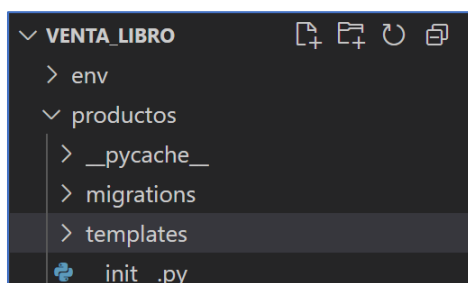
```
productos > views.py > ...
1  from django.http import HttpResponse
2
3  def inicio(request):
4      return HttpResponse("<h1>¡Hola, mundo!</h1>")
5
```

resultado:



Templates HTML

Dentro de la aplicación vamos a crear una carpeta llamada "templates"; es allí donde vamos a crear todos nuestros archivos HTML



Luego, para nuestro ejercicio crearemos una nueva función en nuestro archivo views, y le asignamos una ruta en nuestro archivo urls, ejemplo:

EXPLORER

VENTA_LIBRO

env

productos

__pycache__

migrations

templates

index.html

__init__.py

admin.py

apps.py

models.py

tests.py

urls.py

views.py

urls.py

1 from django.urls import path

2 from .views import inicio, goIndex

3

4 urlpatterns = [

5 path('home/', inicio, name='inicio'),

6 path('index/', goIndex, name='index')

7]

8

9

10

11

12

13

14

15

EXPLORER

VENTA_LIBRO

env

productos

__pycache__

migrations

templates

index.html

__init__.py

admin.py

apps.py

models.py

tests.py

urls.py

views.py

views.py

1 from django.http import HttpResponse

2 from django.shortcuts import render

3

4 def inicio(request):

5 return HttpResponse("<h1>¡Hola, mundo!</h1>")

6

7 def goIndex(request):

8 data = {'usuario': 'Bryan'}

9 return render(request, 'index.html', data)

Debemos tener en cuenta la importación de la funcionalidad **render** de la librería `django.shortcuts`, esto nos ayudara a renderizar las plantillas HTML; también tendremos la opción de enviar data dinámica desde la función hacia la plantilla.

Vamos a crear la plantilla **index.html** a la que apunta nuestra función y renderizamos con la data enviada, para este caso el dato **'usuario'**.

EXPLORER

VENTA_LIBRO

env

productos

__pycache__

migrations

templates

index.html

__init__.py

admin.py

apps.py

index.html

1 <!DOCTYPE html>

2 <html>

3 <head>

4 <title>Página de inicio</title>

5 </head>

6 <body>

7 <h1>Bienvenido, {{ usuario }}</h1>

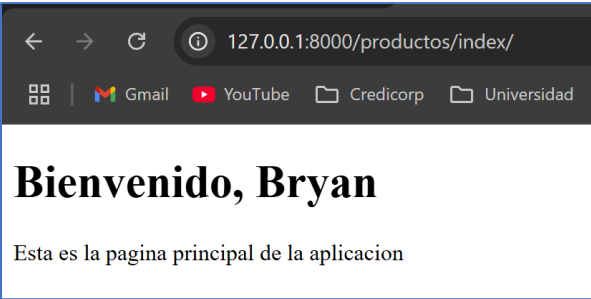
8 <p>Esta es la pagina principal de la aplicacion</p>

9 </body>

10 </html>

11

El resultado sería:



Antes de continuar, debemos entender cómo podemos manipular y recibir la información dentro de las views.

Captura parámetros por URL

Si requerimos de parámetros por medio de la url, debemos modificar la ruta de acceso, para que tenga en cuenta el dato a recibir.

EXPLORER

...

index.html

autor.html

apps.py

views.py 2

urls.py

VENTA LIBRO

env

productos

__pycache__

migrations

templates

autor.html

index.html

productos > urls.py > ...

1 from django.urls import path

2 from .views import inicio, goIndex, parametrosUrl

3

4 urlpatterns = []

5 path('home/', inicio, name='inicio'),

6 path('index/', goIndex, name='index'),

7 path('autor/<int:autor_id>', parametrosUrl, name='autor')

8]

En caso de requerir más de un parámetro, se separan por barra inclinada. En el ejemplo se esta tipando el dato que espera (int), lo cual es una mejor práctica, pero también es posible, aunque no se recomienda no tiparlo.

Recibimos también como parámetro en el view el dato esperado.

apps.py

models.py

tests.py

urls.py

views.py 2

11

12 def parametrosUrl(request, autor_id):

13 data = Autor.objects.get(id=autor_id)

14

15 return render(request, 'autor.html', {'autor': data})

16

Controles de flujo

If, elif, else

Permite mostrar segmentos de la plantilla de acuerdo al cumplimiento de una condición.

1 <!DOCTYPE html>

2 <html>

3 <head>

4 | <title>Página de inicio</title>

5 </head>

6 <body>

7 | {% if autor %}

8 | <h1>Bienvenido, {{ autor.nombre }}</h1>

9 | {% else %}

10 | <h1>Bienvenido querido autor</h1>

11 | {% endif %}

12 | <p>Esta es la pagina principal de la aplicacion</p>

13 </body>

14 </html>

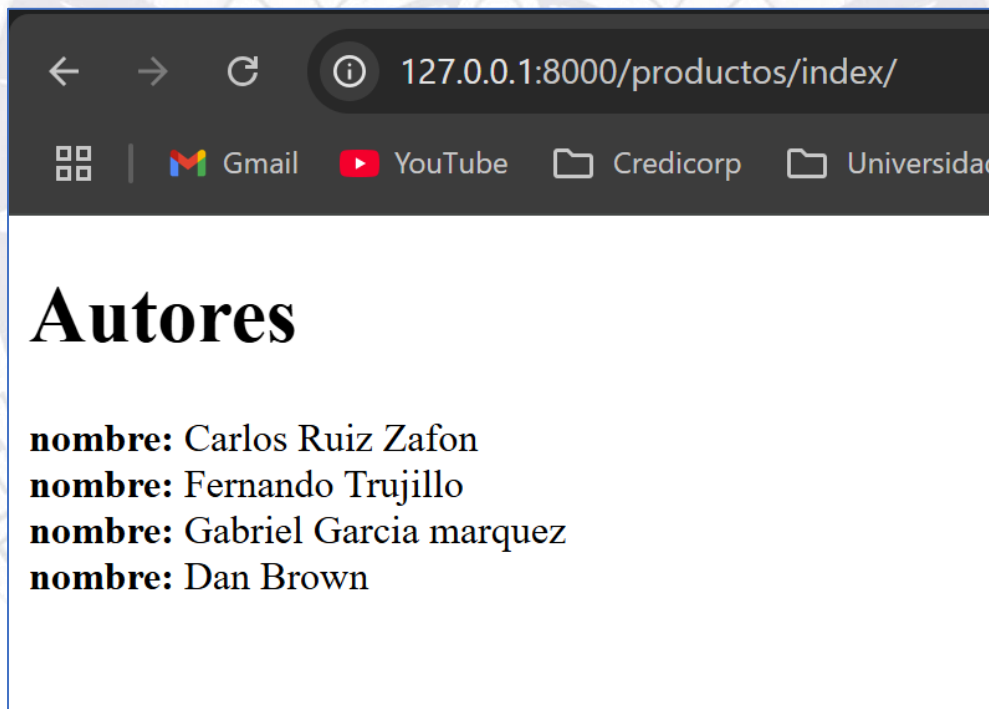
En el anterior ejemplo, en caso de no encontrar ningún objeto autor, mostrara el segmento del else.

For

Permite mostrar segmentos repetitivos de acuerdo a listas existentes. Ejemplo:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Página de inicio</title>
5  </head>
6  <body>
7  |   <h1>Autores</h1>
8  |   {% for autor in autores %}
9  |       <label style="font-weight: bold;"> nombre:</label>
10 |       <label> {{ autor.nombre}} {{ autor.apellido }}</label>
11 |       <br>
12 |   {% endfor %}
13 </body>
14 </html>
```

El resultado sería:



Herencia de plantillas

Block y Extends

Es posible insertar una plantilla dentro de otra utilizando el sistema de herencia de Django. Para ello, es importante entender que existirá una plantilla base o padre, y una o varias plantillas hijas que extenderán su estructura.

La etiqueta `{% block %}` permite definir secciones en la plantilla padre que pueden ser personalizadas o sobrescritas por las plantillas hijas. Así se determina qué partes del contenido pueden variar y dónde se ubicarán.

Por otro lado, la etiqueta `{% extends %}` se utiliza en la plantilla hija para indicar que está heredando la estructura de una plantilla padre, lo que permite reutilizar elementos comunes como encabezados, menús o pies de página.

El block esta compuesto por un nombre que se le quiera otorgar, el extends por su parte esta compuesto por la ruta de la plantilla padre.

```
{% block [nombre] %}

{% extends "[ruta plantilla padre]" %}
```

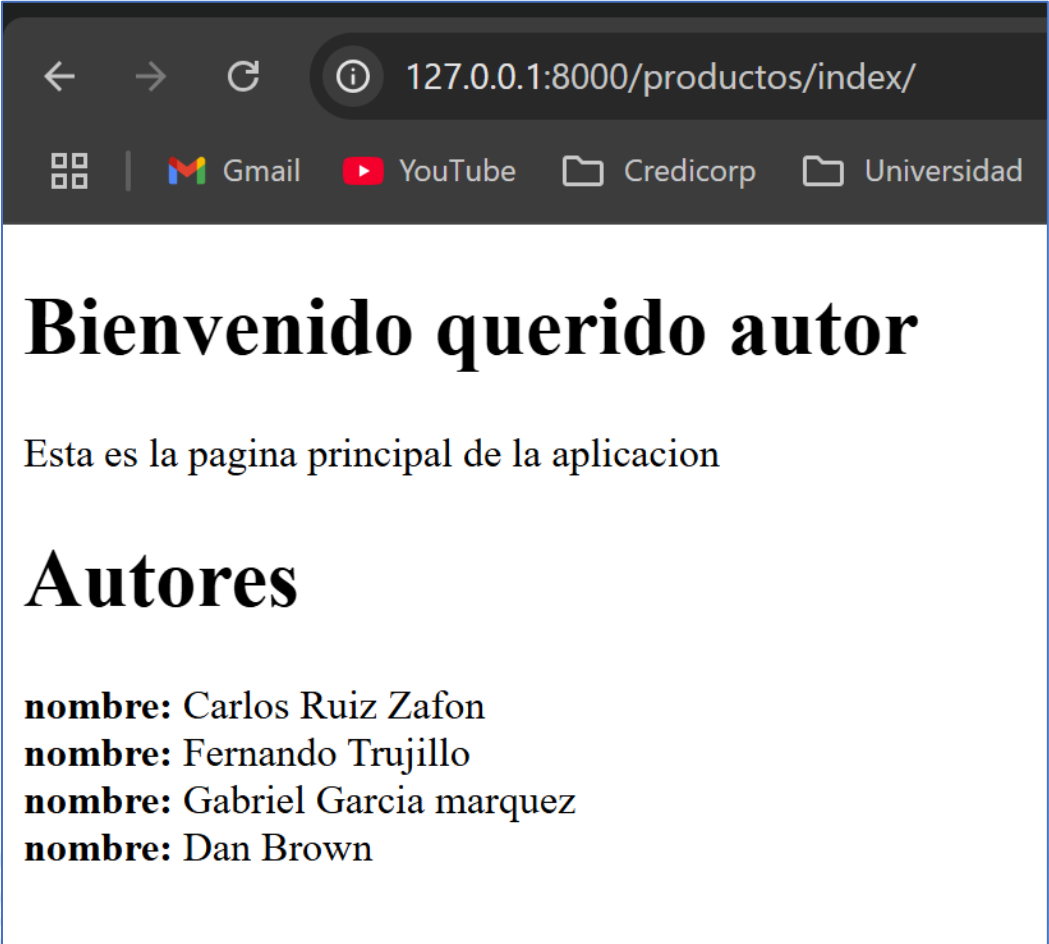
Plantilla padre:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Página de inicio</title>
5  </head>
6  <body>
7  |   {% if autor %}
8  |   |   <h1>Bienvenido, {{ autor.nombre }}</h1>
9  |   {% else %}
10 |   |   <h1>Bienvenido querido autor</h1>
11 |   {% endif %}
12 |   <p>Esta es la pagina principal de la aplicacion</p>
13 |
14 |   {% block lista-autores %}
15 |   {% endblock %}
16 </body>
17 </html>
```

Plantilla hija:

```
1  {% extends "autor.html"%}
2
3  <!DOCTYPE html>
4  <html>
5  <head>
6  |   <title>Página de inicio</title>
7  </head>
8  <body>
9  |   {% block lista-autores %}
10 |   |   <h1>Autores</h1>
11 |   |   {% for autor in autores %}
12 |   |   |   <label style="font-weight: bold;"> nombre:</label>
13 |   |   |   <label> {{ autor.nombre}} {{ autor.apellido }}</label>
14 |   |   |   <br>
15 |   |   {% endfor %}
16 |   {% endblock %}
17 </body>
18 </html>
```

Resultado:



Include

Include también permite la inserción de una plantilla en otra, solo que esta inyecta la plantilla completa (no por secciones block), además es posible pasar variables con la palabra reservada **with**. Ejemplo:

Plantilla padre:

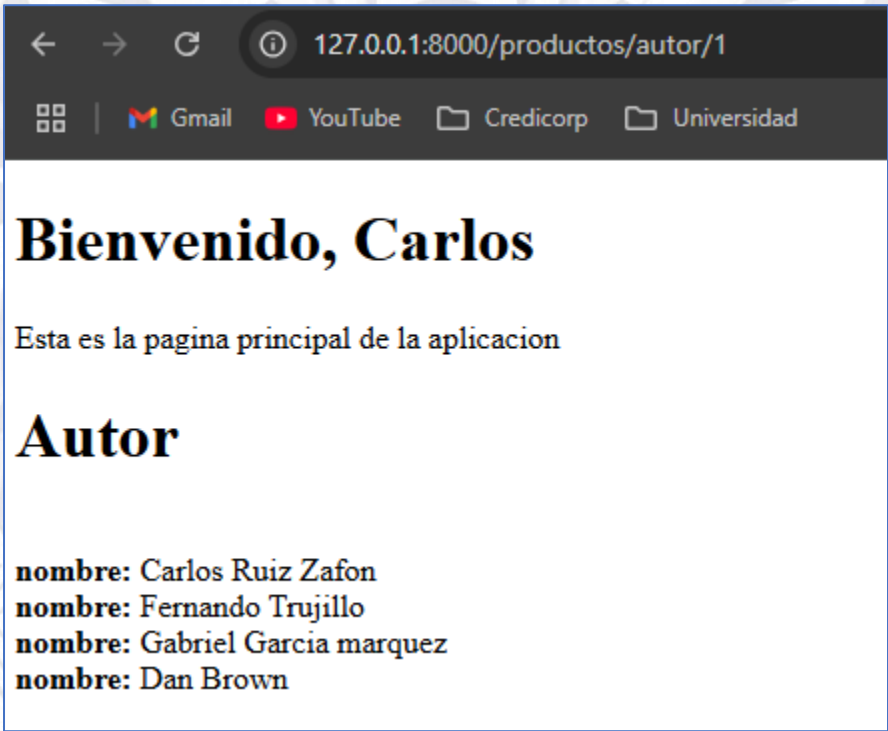
```
<!DOCTYPE html>
<html>
<head>
  <title>Página de inicio</title>
</head>
<body>
  {% if autor %}
    <h1>Bienvenido, {{ autor.nombre }}</h1>
  {% else %}
    <h1>Bienvenido querido autor</h1>
  {% endif %}
  <p>Esta es la pagina principal de la aplicacion</p>

  {% include "index.html" with nombrePadre="Autor"%}
</body>
</html>
```

Plantilla hija:

```
<!DOCTYPE html>
<html>
<head>
|   <title>Página de inicio</title>
</head>
<body>
|   <h1>{{nombrePadre}}</h1>
|   <br>
|   {% for autor in autores %}
|       <label style="font-weight: bold;"> nombre:</label>
|       <label> {{ autor.nombre}} {{ autor.apellido }}</label>
|       <br>
|   {% endfor %}
</body>
</html>
```

Resultado:



Formularios

Para la creación de formularios, se puede realizar la creación de clases encargadas de visualizar la información requerida. Para esto, se debe crear en la raíz del aplicativo un archivo llamado forms.py (podría ser una carpeta que contenga archivos más específicos para proyectos grandes), allí colocamos los campos de referencia, incluyendo sus diferentes tipos.

Es importante mencionar, que la clase creada debe heredar de forms de django, para que este la entienda como un formulario.

```
EXPLORER
VENTA_LIBRO
> env
> productos
  > __pycache__
  > migrations
  > templates
    > autor.html
    > crear_autor.html
    > index.html
  > __init__.py
  > admin.py
  > apps.py
  > forms.py

forms.py
1 from django import forms
2
3 class FormularioAutor(forms.Form):
4     nombre = forms.CharField(label="Nombre", max_length=100)
5     apellido = forms.CharField(label="Apellido", max_length=100)
6     fecha_nacimiento = forms.DateField(label="Fecha Nacimiento",
7                                       widget=forms.DateInput(attrs={'type': 'date'}))
8
9
10
11
12
13
14
```

Luego de esto, se genera la función en el views, en la que se pasa como parámetro el formulario.

```
views.py 9+ X
productos > views.py > ...
30 from django.http import HttpResponse
31 from django.shortcuts import render
32 from .models import Autor
33 from .forms import FormularioAutor
34
35 def crearAutor(request):
36     return render(request, 'crear_autor.html', {'formulario': FormularioAutor})
```

Y el respectivo HTML a visualizar

```
crear_autor.html X
productos > templates > crear_autor.html >
1
2 <form method="POST">
3     {% csrf_token %}
4     {{formulario}}
5     <button>
6         Guardar
7     </button>
8 </form>
```

Determina el formulario, y define el viaje de información por medio del metodo POST.

Medida de proteccion frente a formularios maliciosos. Debe estar en todos los formularios

Aqui es donde se visualiza el formulario creado

Realiza el respective Submit al formulario

En caso de no colocarle el método por el cual los datos viajaran, el sistema tomara por defecto el método GET.

El resultado del todo lo anterior seria:

127.0.0.1:8000/productos/crear-autor/

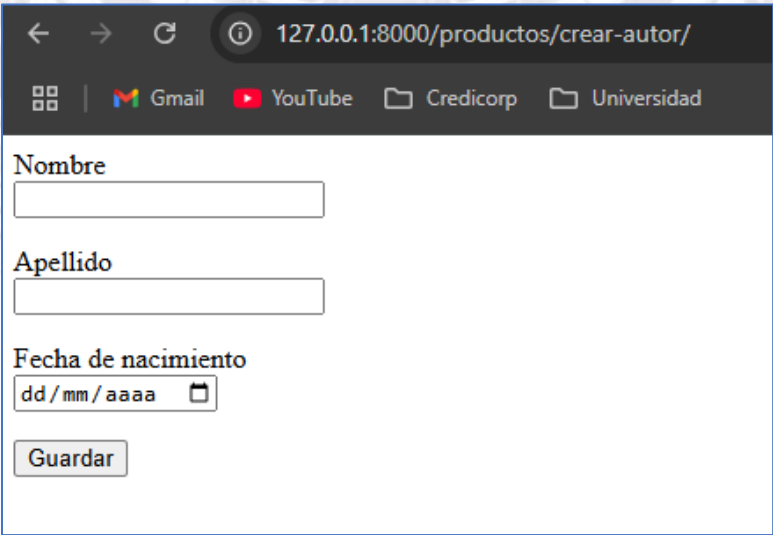
Gmail YouTube Credicorp Universidad

Nombre: Apellido: Fecha Nacimiento:

También es posible, para darle más personalización al estilo, exponer el formulario, llamando cada uno de sus campos por separado.

```
<> crear_autor.html X
productos > templates > <> crear_autor.html > ...
1
2  <form method="POST">
3    {% csrf_token %}
4    <p>
5      <label>Nombre</label>
6      <br>
7      {{formulario.nombre}}
8    </p>
9    <p>
10     <label>Apellido</label>
11     <br>
12     {{formulario.apellido}}
13   </p>
14   <p>
15     <label>Fecha de nacimiento</label>
16     <br>
17     {{formulario.fecha_nacimiento}}
18   </p>
19   <button>
20     Guardar
21   </button>
22 </form>
```

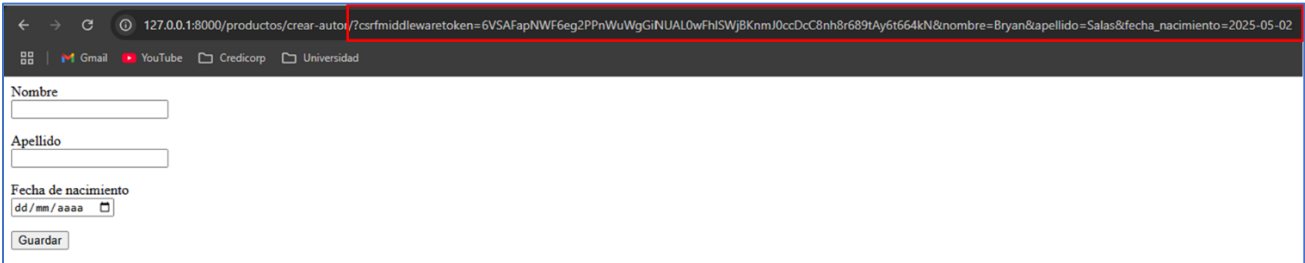
El resultado sería:



Captura parámetros por métodos GET y POST

Entendiendo que el envío de datos del formulario puede hacerse a través de métodos GET o POST, veremos como podemos capturar la información para tratarla en el behind.

Cuando la información viaja a través de métodos GET, podemos observarla directamente en la Url después del signo de cierre de pregunta (?) y la llamamos query params, cada parámetro viene delimitado por su nombre y un valor, además de estar enlazado a otros parámetros por el símbolo ampersand (&).

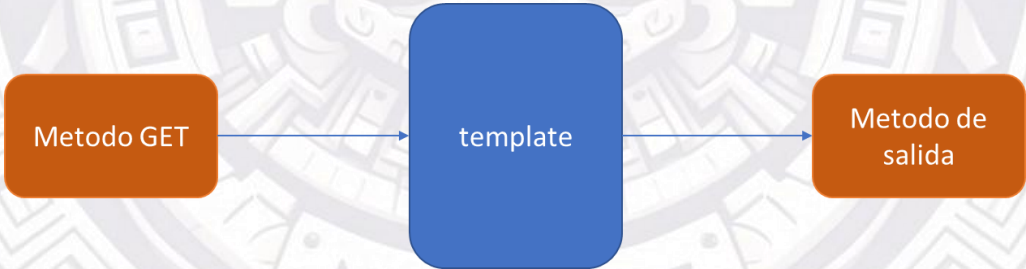


Por el contrario, si la información viaja a través de POST, no es visible.
Una vez en el behind la forma de capturar esta información es:

```
Request.GET['nombre_atributo']  
Request.POST['nombre_atributo']
```

Manejo de formulario en el behind

Todo método cuando es llamado antes de mostrar la interfaz al usuario, se ejecuta por medio del GET, y posterior al hacer submit viajara por otro método(decidido por el desarrollo).



Existen varias formas de trabajar con los formularios enviados hacia el behind, la primera es el envío de la data a otro método que no es el mismo de entrada.

```
crear_autor.html X views.py 9 urls.py  
productos > templates > crear_autor.html > ...  
1  
2 <form method="POST" action="/guardarAutor/">  
3     {% csrf_token %}  
4     <p>  
5         <label>Nombre</label>  
6         <br>  
7         {{formulario.nombre}}  
8     </p>  
9     <p>  
10        <label>Apellido</label>  
11        <br>  
12        {{formulario.apellido}}  
13    </p>  
14    <p>  
15        <label>Fecha de nacimiento</label>  
16        <br>  
17        {{formulario.fecha_nacimiento}}  
18    </p>  
19    <button>  
20        Guardar  
21    </button>  
22 </form>
```

En este caso, a través de la propiedad action del form y usando la url dinámica, viajara a la función guardarAutor al hacer submit sobre el formulario. Esto mismo se puede lograr, usando la propiedad url.

El primer paso es crear un nuevo path en el archivo url.py:

```

<> crear_autor.html  views.py 9  urls.py  X
productos > urls.py > ...
1  from django.urls import path
2  from .views import inicio, goIndex, parametrosUrl, crearAutor, guardarAutor
3
4  urlpatterns = [
5      path('home/', inicio, name='inicio'),
6      path('index/', goIndex, name='index'),
7      path('autor/<int:autor_id>', parametrosUrl, name='autor'),
8      path('crear-autor/', crearAutor, name='crear_autor'),
9      path('guardado-autor/', guardarAutor, name='guardar_autor')
10 ]

```

Posterior se usa la propiedad antes mencionada, en el action del form en la plantilla HTML, utilizando el name para que el sistema encuentre la función (para este caso "guardar_autor").

```

<> crear_autor.html  X  views.py 9  urls.py
productos > templates > <> crear_autor.html > ...
1
2  <form method="POST" action="{% url 'guardar_autor' %}">
3      {% csrf_token %}
4      <p>
5          <label>Nombre</label>
6          <br>
7          {{formulario.nombre}}
8      </p>
9      <p>
10         <label>Apellido</label>
11         <br>
12         {{formulario.apellido}}
13     </p>
14     <p>
15         <label>Fecha de nacimiento</label>
16         <br>
17         {{formulario.fecha_nacimiento}}
18     </p>
19     <button>
20         Guardar
21     </button>
22 </form>

```

Cuando un formulario no incluye la propiedad action, por defecto enviará los datos a la misma URL desde la que fue cargado (generalmente asociada a una vista mediante un método GET). En el ejemplo anterior, el formulario se enviará a la función "crearAutor", por lo que dentro de esa vista será necesario verificar el método de la solicitud (GET o POST) para distinguir si se trata de la primera carga del formulario o del envío de datos (submit).

```

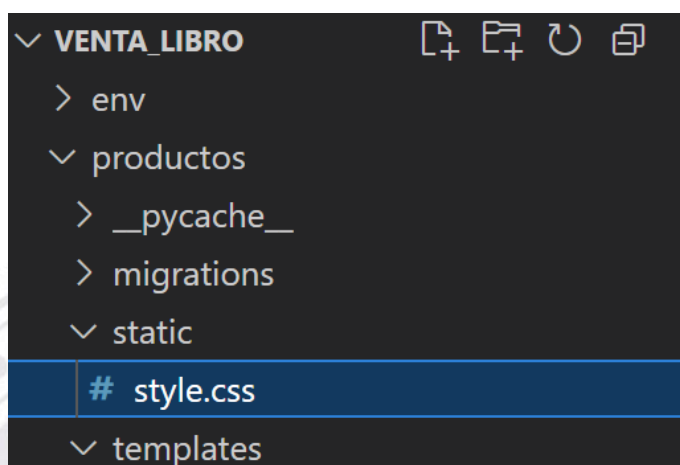
def crearAutor(request):
    if request.method == 'GET':
        print('Esta es la primera vez que ingresa al formulario')
        return render(request, 'crear_autor.html', {'formulario': FormularioAutor})
    else:
        print('La informacion del formulario ya viajo a la funcion a traves del submit')
        print('Debo hacer el guardado de la informacion')
        return redirect('lista_autores')

```

Archivos estáticos

Los templates al contener información variante se cargan de manera dinámica de acuerdo a la interpretación del código en ellos. Sin embargo, es posible referenciar archivos que sean estáticos (no variantes), esto es especialmente útil para imágenes u hojas de estilos.

Para esto, se debe crear la carpeta "static" en la raíz de la aplicación, y allí incluir todos estos archivos necesarios en el código.



Luego, para utilizar estos archivos en una plantilla, es necesario incluir la línea

```
{% load static %}
```

la cual indica que se hará uso de recursos estáticos. A continuación, se deben referenciar los archivos requeridos utilizando la etiqueta

```
{% static 'ruta del archivo.ext' %}
```

dentro de los atributos correspondientes, como href o src.

```
{% load static %}
<link rel="stylesheet" href="{% static style.css %}">

<form method="POST">
    {% csrf_token %}
    {{formulario}}
    <button>
        Guardar
    </button>
</form>
```